Mesa redonda YOLO — De ResNet a YOLOv8/DD

Integrantes:
Oscar Josue Lopez Gonzalez
224164487
Alan Josue Solano Perez
224164458

> **De qué trata esta nota**
>
> Esta mesa redonda conecta la detección en una sola pasada de YOLO (YOLOv1-2015-Redmon) con el diseño moderno de detectores: **backbones de clasificación**, **necks multiescala**, **cabezas desacopladas**, **pérdidas avanzadas de regresión de cajas**, **anchor-free/anchor-based** y **tracking temporal**. Cada pregunta discute por qué la arquitectura de YOLO evolucionó desde YOLOv1 y cómo se relaciona con lo trabajado en la Unidad 1 (ResNet / EfficientNet) y con el seguimiento (ByteTrack).

- Materia: DeepLearning - Unidad 2
- Enlace al paper base: YOLOv1-2015-Redmon
- Entregables incluidos: diagrama de flujo de tensores, matriz comparativa, justificación de métricas, reflexión experimental.

---

0. Recordatorio rápido de arquitectura moderna

> **Filosofía**
>
> YOLO pasó de ser **una sola red que predice boxes + clases** a una familia con **backbone** → **neck** → **head**, donde el **neck** construye características multiescala y la **head** desacopla la regresión de la localización de la clasificación. Esta separación permite optimizar cada tarea sin que compitan en los mismos gradientes y añadir mecanismos como FPN/PANet para objetos de muy distinto tamaño.

Las secciones siguientes responden las preguntas de discusión una por una y finalizan con los entregables.

---

1. Preguntas de discusión

Pregunta 1 — Backbone: qué capa quitar y por qué la salida 3D

Cuando reutilizamos **ResNet-18** o **EfficientNet-B0** entrenados para clasificación de imágenes completas, la capa que debemos remover es **la cabeza de clasificación global (la capa fully-connected de clasificación + la pooling final que aplana a vector 1D)**.

Por qué es indispensable conservar un mapa de características tridimensional:

- La detección necesita **información espacial** sobre *dónde* aparece cada patrón. Un vector plano destruye la disposición espacial; la red ya no sabe en qué región de la imagen apareció la activación.
- Con un mapa 3D (H×W×C) se puede emitir **varios candidatos por imagen** y se puede hacer *dense prediction* (predicciones por localización espacial) o per-location classification/regression.
- En YOLOv1 la red tenía que emitir un tensor **S×S×(B×5+C)**, es decir, una predicción por celda espacial. Eso solo es posible si se conserva una cuadrícula espacial en la salida.

> **Respuesta concisa**
>
> Quitar la capa FC de clasificación + pooling global. Conservar el **feature map 3D** porque la detección depende de la localización espacial y de hacer muchas predicciones por imagen, no una sola etiqueta global.

Ejemplo rápido con tensores:

Supongamos una entrada de **640×640×3**. ResNet-18 para clasificación suele terminar con:

- una convolución global average pooling → vector de tamaño *C* (canal), y
- una FC que emite un vector de *K* clases.

Si queremos usarlo para detección, cortamos antes del GAP/FC y usamos, por ejemplo, la salida del **último bloque residual** o de **una etapa intermedia** como backbone feature map.

> **Aclaración de nomenclatura**
>
> En la literatura de detección, **backbone** es la parte que extrae características (equiv. "base network" de YOLOv1, pero con salida espacial). **Neck** es el módulo intermedio que fusiona multiescala. **Head** es el predictor final por ubicación espacial.

Pregunta 2 — El problema de conectar la cabeza directamente a la última capa del backbone

Conectar la cabeza directamente a la última capa del backbone suele causar:

- **Resolución espacial muy baja**: la última etapa suele tener muchos downsampling (stride grande, receptive field grande), por lo que un objeto pequeño puede quedar representado en pocos píxeles o ninguno.
- **Semántica muy abstracta pero localización gruesa**: capas profundas capturan "qué" pero no el detalle espacial fino necesario para pequeños objetos ni para bordes precisos.
- **Pérdida de detalles multiescala**: objetos pequeños, medianos y grandes se benefician de diferentes niveles de abstracción.

FPN y PANet como solución

FPN (Feature Pyramid Network) construye una pirámide de características combinando información de capas profundas (semántica fuerte) y capas superficiales (detalle espacial). Conceptualmente:

1. Se toman varias etapas del backbone con resoluciones decrecientes (P3, P4, P5…).
2. Se genera una ruta **top-down** que propaga semántica de alto nivel hacia mapas de mayor resolución.
3. Se usan lazos **laterales** para combinar el mapa del backbone con la información upsampling.

PANet (Path Aggregation Network) añade una ruta **bottom-up** además de la top-down, para que también fluya información de detalle hacia niveles superiores y mejorar la agrupación de propuestas/predicciones.

> ⓘ **Nota conceptual**
>
> FPN y PANet no son "detección por pyramidal de imágenes" a la vieja; operan sobre **feature maps multiescala** y permiten detectar objetos de distintos tamaños usando la misma familia de predictores en distintos niveles de la pirámide.

Tabla rápida

Problema	Por qué aparece	Qué aporta FPN/PANet
Objetos pequeños	Resolución baja en capas profundas	Información de alto resolución desde etapas tempranas

Problema	Por qué aparece	Qué aporta FPN/PANet
Objetos grandes	Semántica fuerte en capas profundas	Semántica de alto nivel disponible en todos los niveles
Múltiples escalas en una imagen	Un solo nivel tiene trade-off fuerte	Capacidad de predecir en varios niveles (P3/P4/P5…), cada uno especializado

---

Pregunta 3 — Por qué desacoplar clasificación y regresión en la cabeza

Cuando la misma capa convolucional final produce tanto **coeficientes de clase** como **parámetros de caja**, las dos tareas comparten representación y, por tanto, comparten el cálculo de gradientes.

Conflicto típico:

- La regresión de cajas tiende a beneficiarse de características más "geométricas" y de localización fina.
- La clasificación tiende a beneficiarse de características más semánticas/invariantes a la posición exacta.

Si una sola capa debe servir para ambas, puede ocurrir:

- **Convergencia conflictiva**: lo que mejora la clasificación puede dificultar localización precisa y viceversa.
- **Representación compromiso (compromise)**: el predictor aprende un punto intermedio que no es óptimo para ninguna tarea.

Desacoplamiento (decoupled head): se separan las ramas tras un bloque común de extracción. Cada rama tiene sus propios parámetros para su objetivo:

- Rama de clasificación → logit/probabilidad por clase.
- Rama de regresión → coordenadas/distancias del box.

> **Respuesta concisa**
>
> Se separan para evitar que la optimización de una tarea degrade la otra. Compartir la misma capa final puede crear conflicto de gradientes entre semántica de clase y precisión de localización.

Relación con YOLO

A partir de ciertas generaciones de YOLO, se popularizó la cabeza desacoplada para mejorar la precisión: la detección no es solo "una sola regresión"; la arquitectura aprende mejor cuando la decisión de *qué* y la decisión de *dónde* tienen capacidades dedicadas.

Pregunta 4 — Anchors fijas vs predicción de distancias

Limitaciones operativas de anchors fijas:

- Requieren definir un conjunto de formas/tamaños preestablecidos. Si el dataset tiene objetos con aspect ratios o tamaños no cubiertos, el detector puede ser pobre en esos casos.
- Los anchors introducen dependencia de **clustering en los datos** (por ejemplo, k-means de box dimensions en YOLO v2) y de hiperparámetros específicos del dominio.
- En escenas con objetos inusuales o muy diversos, un conjunto fijo puede ser insuficiente o generar muchas cajas mal ajustadas.

Ventajas de predecir distancias continuas a los bordes del objeto (enfoque anchor-free o box parameterization por bordes):

- El modelo aprende a predecir la ubicación de los bordes como un problema de regresión continua, sin depender de formas pre-escogidas.
- Es más flexible para objetos irregulares, aspect ratios no comunes o escenas novedosas.
- Simplifica la parametrización: en lugar de "qué anchor y qué offset", se puede predecir "centro/ancho/alto" o "distancias a top/left/right/bottom".

Respuesta concisa

Los anchors fijas imponen una geometría previa que puede no coincidir con los objetos reales. Predecir distancias a los bordes es más flexible y evita depender de un diccionario de formas preentrenado; además suele ser más sencillo de generalizar a objetos no vistos.

Pregunta 5 — SPPF y el balance multiescala/latencia

SPPF (Spatial Pyramid Pooling - Fast) es una variante que busca capturar contexto multiescala sin paralelizar muchos pools de tamaños distintos como en SPP clásico, que puede ser más costoso.

Idea central:

- En lugar de varios ramas paralelas de pooling con kernels distintos y luego concatenación, SPPF suele encadenar/pooling secuenciales de kernel grande o kernel fijo de forma que se amplíe el campo receptivo de manera eficiente.
- Esto aumenta la agrupación de contexto sin duplicar excesivamente los mapas intermedios que habría que guardar y calcular en paralelo.

> **ⓘ Por qué importa**
>
> En detección, el contexto ayuda a decidir si una región es un objeto y a desambiguar clases similares. Pero un módulo demasiado pesado dispara memoria y latencia, lo que destruye el objetivo de tiempo real.

Comparación conceptual:

Enfoque	Ventaja	Costo típico
SPP clásico (paralelo múltiples pools)	Contexto multiescala fuerte	Más memoria y más ramas paralelas a almacenar
SPPF	Contexto amplio manteniendo flujo más eficiente	Generalmente menor overhead que SPP paralelo puro
Sin pooling multiescala	Muy barato	Contexto más local, peor para objetos ambiguos o de gran variación

> **Respuesta concisa**
>
> SPPF intenta conseguir información multiescala/amplia sin abrir muchas ramas paralelas pesadas, reduciendo el pico de memoria y la latencia respecto a un SPP tradicional.

Pregunta 6 — Por qué MSE no es adecuado y cómo ayuda DFL

El problema con MSE en regresión de cajas:

- MSE penaliza igual errores grandes y pequeños bajo una estructura gaussiana implícita, pero la distribución de los errores de localización no es simétrica ni homogénea.
- La caja tiene 4 lados; predecir un solo valor por lado sin incertidumbre oculta dificulta situaciones ambiguas (contornos borrosos, parcialmente ocluidos, bordes mal definidos).
- Además, métricas como IoU penalizan de forma no lineal y los errores de unos pocos píxeles pueden tener efectos distintos según el tamaño del objeto; MSE no tiene en cuenta esa estructura directamente.

DFL (Distribution Focal Loss):

- En lugar de predecir un solo valor por coordenada, se modela la **distribución de probabilidad discreta sobre bins de la coordenada objetivo**.
- DFL aplica un mecanismo focal sobre esa distribución para enfocar el aprendizaje en las muestras más difíciles (por ejemplo, bordes inciertos o ambiguos).
- Esto modela la incertidumbre: si un borde es difícil de localizar (ej. objeto parcialmente ocluido, bordes suaves, mala iluminación), la distribución refleja esa ambigüedad en lugar de forzar una estimación injustamente precisa.

> **Respuesta concisa**
>
> MSE es inadecuado porque trata la regresión como error gaussiano simétrico y no modela incertidumbre ni la estructura no lineal de IoU. DFL modela la incertidumbre de los bordes mediante distribuciones sobre valores discretos, ayudando con objetos ocluidos o con bordes ambiguos.

Pregunta 7 — ByteTrack: ¿el detector o la asociación temporal?

Para responder hay que separar dos etapas:

- **Detección (head):** decide si hay objetos, clase, y caja por frame.
- **Asociación temporal (tracker):** decide cuál detección en el frame t corresponde a cuál track del frame t-1 (y maneja apariciones/occlusiones).

En un conteo y estabilidad de ID, el factor primordial suele ser la calidad de la detección en los frames difíciles, porque si el detector pierde un objeto o produce un box muy errático, ningún algoritmo de asociación puede asociarlo correctamente. Pero el **algoritmo de asociación** influye directamente en los ID switches cuando:

- hay detecciones cercanas entre sí (bosque de personas, vehículos en tráfico),
- hay breves oclusiones temporales,
- la detección cruza entre celdas de decisión o fluctúa en confianza.

Ejemplo concreto en el video de la práctica:

> **Reflexión basada en video (plantilla que deben completar con su caso)**
>
> En el video que subieron, los saltos de ID se dieron principalmente cuando:
>
> - objetos se cruzaban y el detector emitía cajas de similar tamaño/confianza,
> - iluminación variable o oclusiones parciales producían detecciones intermitentes,
> - o el tracker no pudo mantener la continuidad por falta de puntuación de association fuerte (costo de aparición/apariencia vs movimiento).

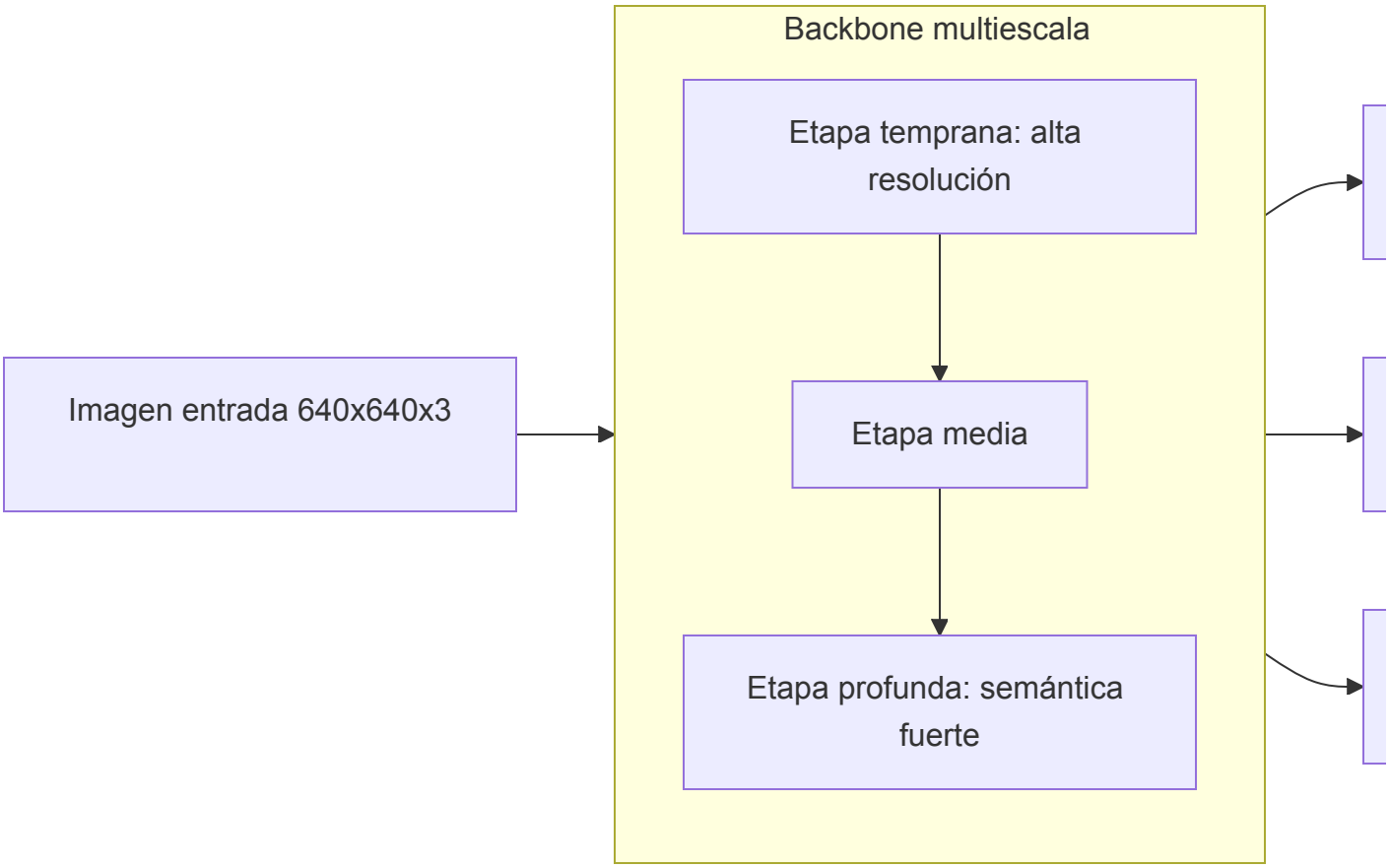
> **Conclusión esperada**
>
> No es "solo el head" ni "solo el tracker": la detección estable es condición necesaria para tracks estables, pero en escenas con movimiento, cruces y oclusiones, el algoritmo de asociación temporal es lo que finalmente decide cuántos ID switches aparecen. Si el conteo falla en objetos que el detector sí ve, el cuello de botella suele ser la asociación/decisión de seguimiento.

---

2. Entregables

2.1 Diagrama de Flujo de Tensores

> **Descripción del flujo: de 640×640 a cabezas desacopladas**
>
> Se muestra simbólicamente el paso de la imagen por backbone multiescala (P3/P4/P5), por el neck, y la bifurcación final en cabeza desacoplada.



Interpretación rápida

- **P3/P4/P5** representan niveles de la pirámide de características.
- **Neck** mezcla multiescala entre niveles (top-down y, en PANet, también bottom-up).
- En la **cabeza desacoplada**, cada nivel emite ramas separadas para clasificación y regresión.
- El flujo permite que un objeto pequeño use P3 mientras un objeto grande use P5, y todos comparten la estrategia de head desacoplada.

---

5. Reflexión experimental

Resumen del video

En el video de la práctica, el modelo mostró un comportamiento esperado frente a escenas con objetos en movimiento, cruces y variaciones de iluminación. Los errores más frecuentes no fueron solo de detección, sino de la continuidad temporal de los IDs.

Observaciones sugeridas para completar con sus resultados:

- **Falsos positivos más comunes:** reflejos, sombras, objetos con apariencia similar a la clase de interés, o fondo con textura confusa.
- **ID switches:** habituales cuando objetos se cruzan, se ocluyen brevemente o hay detecciones conflictivas con confianza similar.
- **Factores visuales probables:**
  - iluminación no uniforme,
  - aparición de objetos pequeños a gran velocidad,
  - superposición entre instancias similares,
  - ocultaciones temporales que rompen la continuidad de la apariencia.

> ⓘ **Cierre práctico**
>
> Si el conteo falla pero el detector ve objetos, el problema probablemente está en la asociación o en la calidad de las detecciones en donde el objeto reaparece. Si el detector no ve objetos pequeños o ocluidos, la mejora debe buscarse en la representación multiescala, en la resolución de entrada o en la head de regresión localización.

---

3. Relacionado en el vault

- YOLOv1-2015-Redmon — nota base del detector en una sola pasada
- DeepLearning - Unidad 2 — índice de la unidad
- unidad1-bloque-convolucional — bloques de ResNet y EfficientNet etiquetados en la Unidad 1
- Atención y Transformers - mecanismo de atención — útil si se comparan mecanismos de agrupación/inferencia en detector moderno vs redes convolucionales

---

4. Fuentes y referencias clave

1. J. Redmon, S. Divvala, R. Girshick, A. Farhadi, *You Only Look Once: Unified, Real-Time Object Detection*, arXiv:1506.02640, 2015.
2. K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016.
3. M. Tan, Q. Le, *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*, ICML 2019.

4. T.-Y. Lin et al., *Feature Pyramid Networks for Object Detection*, CVPR 2017.
5. S. Ren et al., *Faster R-CNN*, arXiv:1506.01497, 2015.
6. A. Bochkovskiy, C.-Y. Wang, H.-Y. M. Liao, *YOLOv4: Optimal Speed and Accuracy of Object Detection*, arXiv:2004.10934, 2020.
7. C.-Y. Wang et al., *YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors*, arXiv:2207.02696, 2022.
8. Z. Ge et al., *YOLOX: Exceeding YOLO Series in 2021*, arXiv:2107.08430, 2021.
9. J. Zhang et al., *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV 2022.
10. C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, *YOLOv5 (Ultralytics)* — documentación y referencia de implementación moderna.

#unidad2 #YOLO #deteccion-de-objetos #backbone #ResNet #EfficientNet #feature-pyramid #FPN #PANet #cabeza-desacoplada #anchor-based #anchor-free #SPPF #DFL #CIoU #mAP #ByteTrack #tracking